

NIKE SALES

SQL REPORT

DESCRIPTION FOR NIKESALES

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the 'NikeSales' database selected. The right pane shows a SQL query window with the following query:

```
SELECT
1  COUNT(*) AS Total_Transactions,
2  SUM(Quantity) AS Total_units_sold,
3  SUM(Gross_Sales_USD) AS Sales_Before_Discount,
4  SUM(Net_Sales_USD) AS Sales_After_Discount,
5  SUM(Order_Total_USD) AS Total_Revenue,
6  AVG(Order_Total_USD) AS Average_Order
7  FROM dbo.Transactions;
```

The query results are displayed in a table with the following columns: Total_Transactions, Total_units_sold, Sales_Before_Discount, Sales_After_Discount, Total_Revenue, and Average_Order. The results show 15090 transactions, 17537 units sold, and a total revenue of 1941215.51.

Total_Transactions	Total_units_sold	Sales_Before_Discount	Sales_After_Discount	Total_Revenue	Average_Order
15090	17537	1764933.00	1666667.95	1941215.51	122.7543

SELECT TOP 10 PRODUCT

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the Object Explorer with the 'NikeSales' database selected. The right pane shows a SQL query window with the following query:

```
SELECT TOP 10
1  Product_Name,
2  SUM(Quantity) AS Total_Units_Sold,
3  SUM(Gross_Sales_USD) AS Sales_Before_Discount,
4  SUM(Net_Sales_USD) AS Sales_After_Discount,
5  SUM(Order_Total_USD) AS Total_Revenue
6  FROM dbo.Transactions
7  GROUP BY Product_Name
8  ORDER BY Total_Revenue DESC;
```

The query results are displayed in a table with the following columns: Product_Name, Total_Units_Sold, Sales_Before_Discount, Sales_After_Discount, and Total_Revenue. The results show the top 10 products by total revenue.

Product_Name	Total_Units_Sold	Sales_Before_Discount	Sales_After_Discount	Total_Revenue
Jordan Essentials Series 6	313	67608.00	63980.04	69795.62
Jordan Flight Series 10	293	67097.00	63556.66	69391.27
Jordan Flight Series 5	274	58636.00	55916.08	61055.37
Jordan Essentials Series 12	303	57873.00	55488.31	60769.54
Air Jordan Series 4	286	58014.00	53706.12	58665.62
Pegasus 30	310	55900.00	52995.60	57871.76
Jordan Sport Series 1	286	55484.00	52605.04	57514.03
Jordan Essentials Series 11	307	55240.00	52676.80	57326.11
Pegasus 25	335	54605.00	51527.56	56453.36
Jordan 1 Series 9	287	53400.00	51028.20	55721.45

TOTAL REVENUE FOR EACH REGION

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'NikeSales'. The central query window contains the following SQL code:

```
1 SELECT Region,
2 COUNT(DISTINCT Order_ID) AS Total_Orders,
3 SUM(Net_Sales_USD) AS Net_Sales,
4 SUM(Order_Total_USD) AS Total_Revenue
5
6 FROM dbo.Transactions
7
8 GROUP BY Region
9 ORDER BY Total_Revenue DESC;
```

The Results pane at the bottom displays the query output as a table with 5 rows and 4 columns: Region, Total_Orders, Net_Sales, and Total_Revenue.

Region	Total_Orders	Net_Sales	Total_Revenue
EMEA	4066	452345.96	439627.27
North America	3617	462270.91	444504.43
APLA	2863	361966.65	333404.44
Latin America	2327	254298.09	293030.12
Greater China	2307	253889.14	280749.25

SELECT TOP 10 CUSTOMERS

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'NikeSales'. The central query window contains the following SQL code:

```
1 SELECT TOP 10
2 Customer_ID, Customer_Name, Customer_Segment,
3 COUNT(DISTINCT Order_ID) AS Total_Orders,
4 SUM(Gross_Sales_USD - Net_Sales_USD) AS Total_Discount_Amount,
5 SUM(Net_Sales_USD) AS Sales_After_Discount,
6 SUM(Order_Total_USD) AS Total_Revenue
7
8 FROM dbo.Transactions
9
10 GROUP BY Customer_ID, Customer_Name, Customer_Segment
11 ORDER BY Total_Revenue DESC;
```

The Results pane at the bottom displays the query output as a table with 10 rows and 7 columns: Customer_ID, Customer_Name, Customer_Segment, Total_Orders, Total_Discount_Amount, Sales_After_Discount, and Total_Revenue.

Customer_ID	Customer_Name	Customer_Segment	Total_Orders	Total_Discount_Amount	Sales_After_Discount	Total_Revenue
C001548	Emma Jones	Member	18	59.1300	2688.87	2956.25
C001162	Casey Smith	Member	20	159.1700	2535.63	2771.60
C000434	Avery Thomas	Business	15	91.3200	2371.68	2605.13
C001333	Alex Miller	Member	16	57.6200	2295.38	2568.35
C000268	Noah Williams	Consumer	15	232.8600	2261.14	2452.39
C000070	Leo Young	Member	17	151.7000	2137.30	2369.00
C000118	Olivia Walker	Business	12	144.3000	2153.70	2339.62
C001000	Alex Anderson	Student	16	77.5200	2087.49	2339.47
C001795	Zoe Davis	Business	15	82.0900	2098.91	2269.36
C000391	Piley Moore	Business	12	89.3800	2037.70	2252.93

DISCOUNT FOR TOP 10 CUSTOMERS

SQL Query1...nourm (66)*

```
1 SELECT TOP 10
2 Customer_ID,
3 Customer_Name,
4 Customer_Segment,
5 SUM(Gross_Sales_USD - Net_Sales_USD) AS Total_Discount_Amount,
6 FORMAT(SUM(Gross_Sales_USD - Net_Sales_USD) / SUM(Gross_Sales_USD), '0.00%') AS Average_Discount_Percentage
7
8 FROM dbo.Transactions
9
10 GROUP BY Customer_ID, Customer_Name, Customer_Segment
11 ORDER BY Total_Discount_Amount DESC;
```

Results Messages

Customer_ID	Customer_Name	Customer_Segment	Total_Discount_Amount	Average_Discount_Percentage
C000227	Cameron Jones	Business	250.9500	13.35%
C001193	Lucas Garcia	Consumer	244.7500	11.52%
C000369	Neah Williams	Consumer	232.8600	9.34%
C001722	Olivia Brown	Member	218.8900	18.99%
C000287	Cameron Allen	Member	216.7400	11.09%
C000890	Taylor Hall	Student	209.6800	10.99%
C000375	Leo Allen	Consumer	193.0200	11.62%
C000784	Zoe Miller	Business	186.1200	12.09%
C000613	Morgan Anderson	Member	185.1900	17.36%
C001322	Mia Jones	Business	181.4100	12.01%

Query executed successfully.

CUSTOMER SEGMENT NET SALES

SQL Query1...nourm (66)*

```
1 SELECT
2 Customer_Segment,
3 COUNT(DISTINCT Customer_ID) AS Unique_Customers,
4 COUNT(DISTINCT Order_ID) AS Total_Orders,
5 SUM(Net_Sales_USD) AS Total_Net_Sales,
6 AVG(Net_Sales_USD) AS Avg_Sales
7 FROM dbo.Transactions
8 GROUP BY Customer_Segment
9 ORDER BY Total_Net_Sales DESC;
```

Results Messages

Customer_Segment	Unique_Customers	Total_Orders	Total_Net_Sales	Avg_Sales
Student	473	2852	435086.94	109.6149
Member	458	3834	429665.49	112.8871
Business	442	3639	404437.63	111.1703
Consumer	427	3566	397477.89	111.4832

Query executed successfully.

HIGHEST REVENUE PAYMENT METHODS

SQL Query Editor window showing a query executed successfully. The query is:

```
1 SELECT Payment_Method,
2 COUNT(DISTINCT Order_ID) AS Total_Orders,
3 SUM(Order_Total_USD) AS Total_Revenue,
4 AVG(Order_Total_USD) AS Average_Order_Value
5
6 FROM dbo.Transactions
7
8 GROUP BY Payment_Method
9 ORDER BY Total_Revenue DESC;
```

The results table shows the following data:

Payment_Method	Total_Orders	Total_Revenue	Average_Order_Value
Cash	3604	469203.40	122.3447
Card	3625	463957.18	121.2959
Bank Transfer	3680	481033.84	125.2809
Digital Wallet	3691	447121.09	121.1361

Sales Performance by Sales Chanel

SQL Query Editor window showing a query executed successfully. The query is:

```
1 SELECT Sales_Channel,
2 COUNT(DISTINCT Order_ID) AS Total_Orders,
3 SUM(Quantity) AS Total_units_sold,
4 SUM(Order_Total_USD) AS Totalrevenue
5
6 FROM dbo.Transactions
7
8 GROUP BY Sales_Channel
9 ORDER BY Totalrevenue DESC;
```

The results table shows the following data:

Sales_Channel	Total_Orders	Total_units_sold	Totalrevenue
Nike Direct	8467	9919	1057840.11
Wholesale	6533	7618	783475.49

Product Segmentation & Price Tier Analysis

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'localhost (SQL Server 17.0.1000.7)'. The central pane shows a query window with the following SQL code:

```
1 SELECT
2     UPPER(Brand) AS Brand,
3     LOWER(Category) AS Category,
4     AVG(Net_Sales_USD) AS AVG_Net_Sales_USD,
5
6     CASE
7         WHEN AVG(Net_Sales_USD) >= 150 THEN 'High-Value Product'
8         WHEN AVG(Net_Sales_USD) >= 80 THEN 'Mid-Range Product'
9         ELSE 'Budget Product'
10    END AS Product_Tier
11
12 FROM dbo.Transactions
13 GROUP BY Brand, Category
14 ORDER BY AVG_Net_Sales_USD DESC;
```

The Results pane at the bottom shows the output of the query, displaying 5 rows of data:

Brand	Category	AVG_Net_Sales_USD	Product_Tier
JORDAN	jordan	294.1058	High-Value Product
NIKE	footwear	148.3725	Mid-Range Product
CONVERSE	converse	77.1279	Budget Product
NIKE	apparel	73.4471	Budget Product
NIKE	equipment	54.9399	Budget Product

The status bar at the bottom indicates 'Query executed successfully.' and 'localhost (17.0 RTM) LAPTOP-H6LGGQR\nouirm ... NikeSales | 00:00:00 | Row: 1, Col: 1 | 5 rows'.

Men vs. Women Sales Analysis

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'localhost (SQL Server 17.0.1000.7)'. The central pane shows a query window with the following SQL code:

```
1 SELECT
2     Gender,
3     SUM(Quantity) AS Total_Units_Sold,
4     SUM(Net_Sales_USD) AS Total_Sales,
5     SUM(Order_Total_USD) AS Total_Revenue
6 FROM dbo.Transactions
7 WHERE Gender = 'MEN'
8 GROUP BY Gender
9
10 UNION ALL
11
12 SELECT
13     Gender,
14     SUM(Quantity) AS Total_Units_Sold,
15     SUM(Net_Sales_USD) AS Total_Sales,
16     SUM(Order_Total_USD) AS Total_Revenue
17 FROM dbo.Transactions
18 WHERE Gender = 'WOMEN'
19 GROUP BY Gender;
```

The Results pane at the bottom shows the output of the query, displaying 2 rows of data:

Gender	Total_Units_Sold	Total_Sales	Total_Revenue
Men	2234	314347.53	347024.82
Women	2170	232592.00	267516.70

The status bar at the bottom indicates 'No issues found' and 'localhost (17.0 RTM) LAPTOP-H6LGGQR\nouirm ... NikeSales | 00:00:00 | Row: 1, Col: 1 | 2 rows'.

Gender Sales & Discount Impact Analysis

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including the 'NikeSales' database and its tables. The central pane shows a SQL query titled 'SQLQuery1...noum (66)' with the following code:

```
1 SELECT Gender,
2 SUM(Quantity) AS Units_Sold,
3 SUM(Gross_Sales_USD) AS Sales_Before_Discount,
4 SUM(Net_Sales_USD) AS Sales_After_Discount,
5 SUM(Order_Total_USD) AS Total_Revenue
6
7 FROM dbo.Transactions
8 GROUP BY Gender
9 ORDER BY Total_Revenue DESC;
```

The Results pane at the bottom displays the query output as a table with 4 rows and 5 columns:

Gender	Units_Sold	Sales_Before_Discount	Sales_After_Discount	Total_Revenue
Unisex	7360	550106.00	521865.13	581409.83
Kids	3757	523868.00	497042.51	545364.16
Women	3178	358623.00	332562.68	367516.70
Men	3244	320036.00	314347.53	347028.62

The status bar at the bottom indicates 'Query executed successfully.' and 'localhost (17.0 RTM) LAPTOP-I6LGGQR\noum NikeSales 00:00:00 Row: 1, Col: 1 4 rows'.

Customer Segment Revenue & Order Analysis

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure, including the 'NikeSales' database and its tables. The central pane shows a SQL query titled 'SQLQuery1...noum (76)' with the following code:

```
1 SELECT
2 C.Customer_Segment,
3 SUM(T.Net_Sales_USD) AS Total_Revenue,
4 COUNT(T.Customer_ID) AS Total_Orders
5 FROM dbo.Transactions AS T
6 INNER JOIN dbo.DimCustomer AS C
7 ON T.Customer_ID = C.Customer_ID
8 GROUP BY
9 C.Customer_Segment
10 ORDER BY
11 Total_Revenue DESC;
```

The Results pane at the bottom displays the query output as a table with 4 rows and 3 columns:

Customer_Segment	Total_Revenue	Total_Orders
Student	4039394.75	36040
Member	4000319.80	35682
Business	3759662.06	33058
Consumer	3679518.06	32870

The status bar at the bottom indicates 'Query executed successfully.' and 'localhost (17.0 RTM) LAPTOP-I6LGGQR\noum NikeSales 00:00:00 Row: 1, Col: 1 4 rows'.

Regional Order Status & Sales Analysis View

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'NikeSales'. The main query window shows the following SQL code:

```
1 CREATE VIEW vw_RegionOrderStatus AS
2 SELECT
3     Region,
4     Order_Status,
5     COUNT(*) AS Total_Orders,
6     SUM(Quantity) AS Total_Units_Sold,
7     SUM(Gross_Sales_USD) AS Total_Gross_Sales,
8     SUM(Net_Sales_USD) AS Total_Net_Sales
9 FROM dbo.Transactions
10 GROUP BY
11     Region,
12     Order_Status;
```

The Messages pane at the bottom indicates that the commands completed successfully. The completion time is 2026-09-13T16:45:42.1786026+03:00.

The screenshot shows the SQL Server Enterprise Manager interface with the query window displaying the following SQL code:

```
1 SELECT *
2 FROM vw_RegionOrderStatus;
```

The Results pane at the bottom displays the data returned by the query. The data is organized into 10 rows and 5 columns: Region, Order_Status, Total_Orders, Total_Units_Sold, and Total_Gross_Sales. The data is as follows:

Region	Order_Status	Total_Orders	Total_Units_Sold	Total_Gross_Sales
Latin America	Returned	580	678	66922.00
Greater China	Returned	578	680	68899.00
EMEA	Returned	1049	1223	124874.00
Latin America	Delivered	1747	2040	202985.00
North America	Returned	898	1045	104785.00
EMEA	Delivered	3017	3437	35183.00
APLA	Delivered	2035	2387	2436.00
North America	Delivered	2719	3180	31863.00
Greater China	Delivered	1729	2019	198727.00
APLA	Returned	640	768	76867.00

The Messages pane at the bottom indicates that the query executed successfully. The completion time is 2026-09-13T16:45:42.1786026+03:00.

Revenue Analysis by Product Size

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'NikeSales'. The central query window shows the following SQL query:

```
1 SELECT Size
2 SUM(Net_Sales_USD) AS Total_Revenue
3 FROM dbo.Transactions
4
5 GROUP BY Size
6 ORDER BY Total_Revenue DESC;
```

The Results pane at the bottom displays the query output as a table with two columns: 'Size' and 'Total_Revenue'. The data is sorted in descending order of total revenue.

Size	Total_Revenue
XS	325604.69
US 8	276601.65
US 10	245550.01
S	215965.10
M	176986.56
XL	103672.90
US 9	91337.43
L	78612.64
One Size	71238.00
US 7	55768.66
US 11	29432.21

The status bar at the bottom indicates 'Query executed successfully.' and 'Row: 1, Col: 1 11 rows'.

Top Performing Products Ranked Within Categories

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'NikeSales'. The central query window shows the following SQL query:

```
1 SELECT
2 Category,
3 Product_Name,
4 SUM(Net_Sales_USD) AS Total_Revenue,
5
6 RANK() OVER (
7 PARTITION BY Category
8 ORDER BY SUM(Net_Sales_USD) DESC
9 ) AS Product_Rank
10 FROM dbo.Transactions
11
12 GROUP BY
13 Category,
14 Product_Name;
```

The Results pane at the bottom displays the query output as a table with four columns: 'Category', 'Product_Name', 'Total_Revenue', and 'Product_Rank'. The data is grouped by category and ranked by total revenue within each category.

Category	Product_Name	Total_Revenue	Product_Rank
Apparel	Club Place Series 4	31425.44	1
Apparel	Pro Series 10	29796.12	2
Apparel	ACG Series 6	20107.28	3
Apparel	Pro Series 2	18694.00	4
Apparel	Therma-FIT Series 3	18488.48	5
Apparel	Club Place Series 11	18389.12	6
Apparel	Therma-FIT Series 9	17996.75	7
Apparel	Sportswear Series 8	15107.95	8
Apparel	Dri-FIT Series 5	14378.00	9
Apparel	Dri-FIT Series 7	12972.46	10
Apparel	Sportswear Series 1	12805.40	11
Apparel	Club Place Series 12	10903.66	12
Converse	All Star Series 6	25409.28	1
Converse	All Star Series 11	25158.36	2
Converse	Weapon Series 3	22505.90	3
Converse	Chuck 70 Series 9	21410.62	4
Converse	Chuck 70 Series 13	18750.60	5

The status bar at the bottom indicates 'Query executed successfully.' and 'Row: 1, Col: 1 60 rows'.

Month-over-Month (MoM) Revenue Growth Analysis

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure, including the 'NikeSales' database. The central pane shows a SQL query titled 'SQLQuery1...noum (79)' with the following code:

```
1 WITH MonthlySummary AS (  
2     SELECT  
3         FORMAT(Order_Date, 'yyyy-MM') AS Sales_Month,  
4         SUM(Net_Sales_USD) AS Monthly_Net_Sales  
5     FROM dbo.Transactions  
6     GROUP BY FORMAT(Order_Date, 'yyyy-MM')  
7 )  
8 SELECT  
9     Sales_Month,  
10    Monthly_Net_Sales,  
11    LAG(Monthly_Net_Sales, 1) OVER (ORDER BY Sales_Month) AS Previous_Month_Sales,  
12    (Monthly_Net_Sales - LAG(Monthly_Net_Sales, 1) OVER (ORDER BY Sales_Month)) AS Monthly_Growth  
13 FROM MonthlySummary  
14 ORDER BY Sales_Month;
```

The Results pane shows the output of the query, which includes columns for Sales_Month, Monthly_Net_Sales, Previous_Month_Sales, and Monthly_Growth. The data is sorted by Sales_Month, showing a steady increase in net sales over time, with a corresponding positive monthly growth rate.

Sales_Month	Monthly_Net_Sales	Previous_Month_Sales	Monthly_Growth
2025-01	145300.46	NULL	NULL
2025-02	110724.49	145300.66	-2676.17
2025-03	134319.21	110724.49	15594.72
2025-04	140886.00	134319.21	6366.79
2025-05	146821.82	140886.00	5325.12
2025-06	138603.46	146821.82	-7418.36
2025-07	143428.47	138603.46	4825.01
2025-08	133955.66	143428.47	-9472.81
2025-09	140325.53	133955.66	6369.87
2025-10	144984.97	140325.53	4659.44
2025-11	134219.45	144984.97	-10765.52
2025-12	146088.13	134219.45	11878.68

The status bar at the bottom indicates that the query was executed successfully, returning 12 rows.

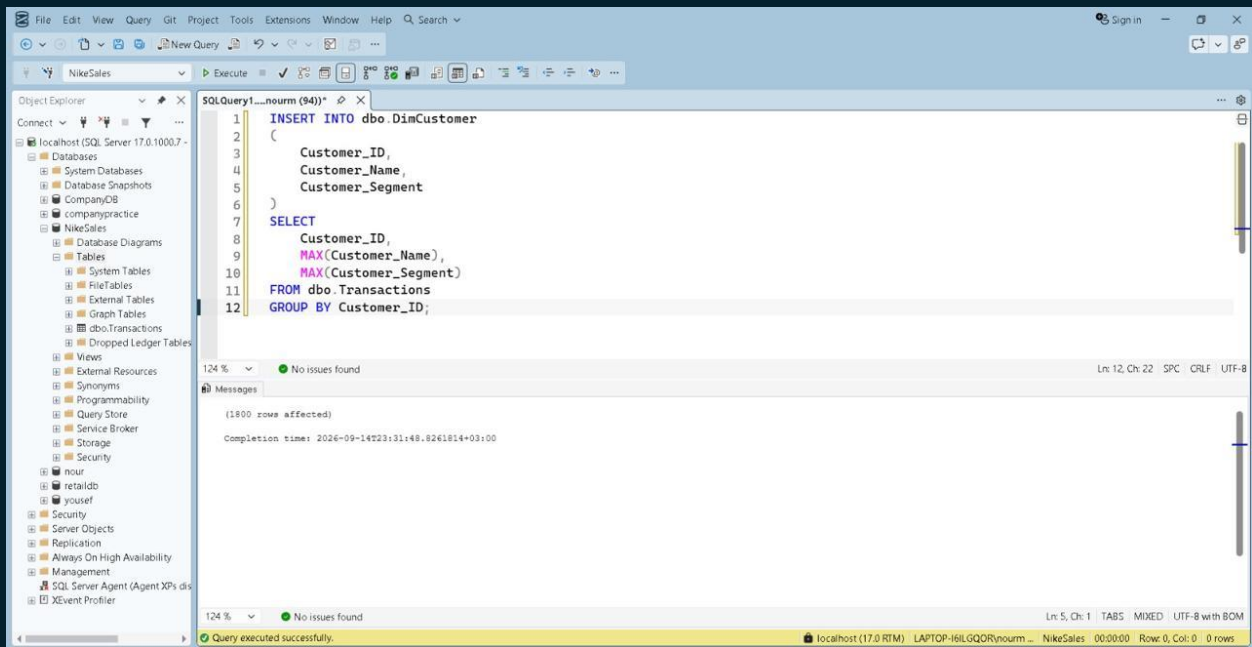
Customer Dimension Table Creation

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure, including the 'NikeSales' database. The central pane shows a SQL query titled 'SQLQuery1...noum (94)' with the following code:

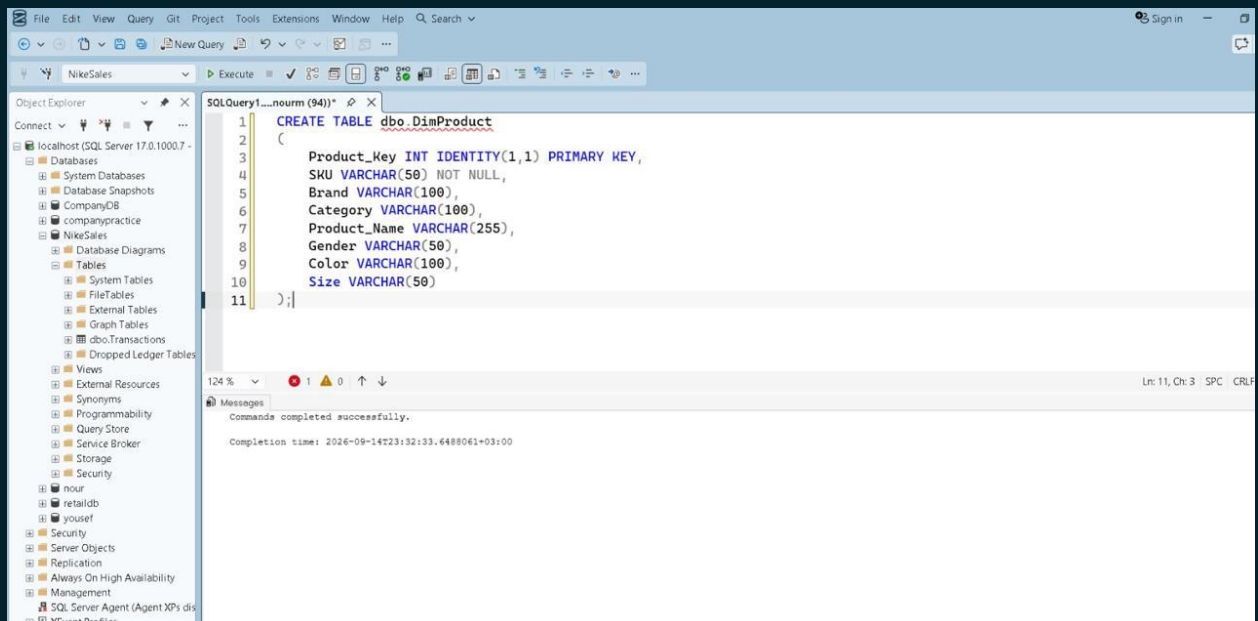
```
1 CREATE TABLE dbo.DimCustomer  
2 (  
3     Customer_Key INT IDENTITY(1,1) PRIMARY KEY,  
4     Customer_ID VARCHAR(50) NOT NULL,  
5     Customer_Name VARCHAR(255),  
6     Customer_Segment VARCHAR(100)  
7 );
```

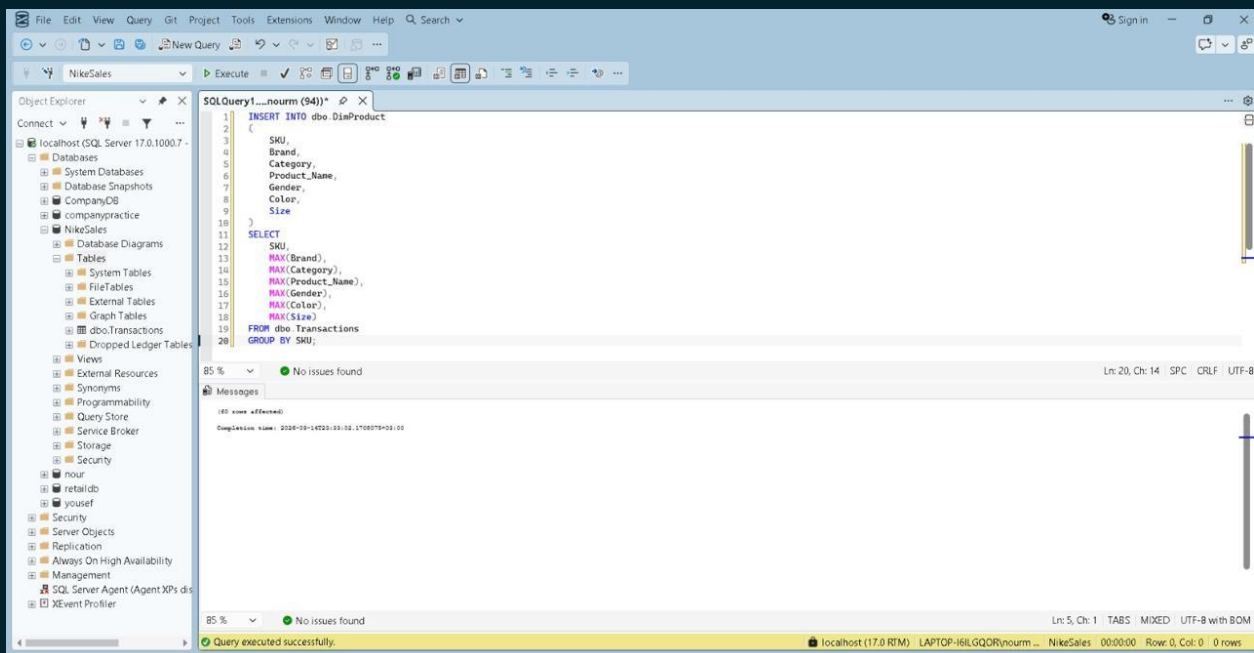
The Messages pane shows the execution results, indicating that the command was completed successfully. The completion time is 2026-09-14T23:30:27.9225262+03:00.

The status bar at the bottom indicates that the query was executed successfully, returning 0 rows.

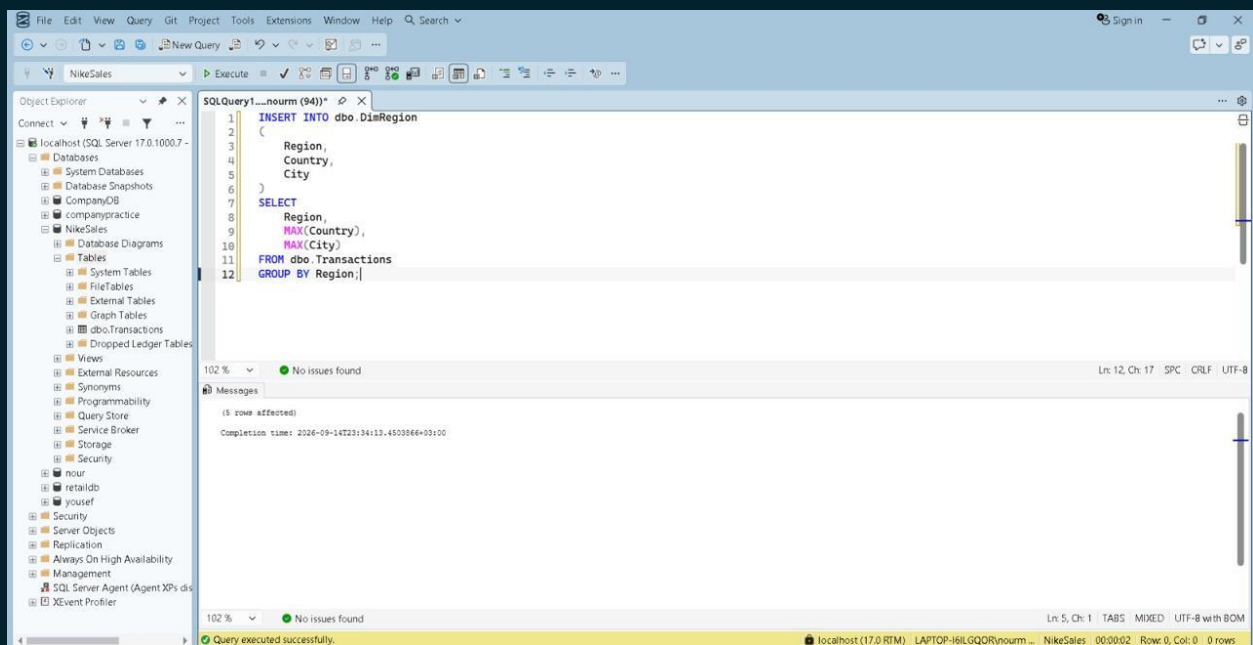


PRODUCT Dimension Table Creation





REGION Dimension Table Creation



DATE Dimension Table Creation

SQL Server Enterprise Edition (64-bit) - Microsoft SQL Server 17.0.10000.7

Object Explorer: Connect to localhost (SQL Server 17.0.10000.7) > Databases > NikeSales

SQLQuery1...nourm (80):

```
1 CREATE TABLE dbo.DimDate
2 (
3     Date_Key INT IDENTITY(1,1) PRIMARY KEY,
4     Order_Date DATE NOT NULL,
5     Year INT,
6     Month INT,
7     Month_Name VARCHAR(20),
8     Quarter INT
9 );
```

Messages: Command completed successfully. Completion time: 2024-09-14T23:36:33.0891639+03:00

102 % No issues found

Ln: 9, Ch: 3 SPC CRLF UTF-8

Query executed successfully. localhost (17.0 RTM) LAPTOP-H8LGQORynourm NikeSales 00:00:00 Row: 0, Col: 0 0 rows

SQL Server Enterprise Edition (64-bit) - Microsoft SQL Server 17.0.10000.7

Object Explorer: Connect to localhost (SQL Server 17.0.10000.7) > Databases > NikeSales

SQLQuery1...nourm (80):

```
1 INSERT INTO dbo.DimDate
2 (
3     Order_Date,
4     Year,
5     Month,
6     Month_Name,
7     Quarter
8 )
9 SELECT DISTINCT
10     CAST(Order_Date AS DATE),
11     YEAR(Order_Date),
12     MONTH(Order_Date),
13     DATENAME(MONTH, Order_Date),
14     DATEPART(QUARTER, Order_Date)
15 FROM dbo.Transactions;
```

Messages: (148 rows affected) Completion time: 2024-09-14T23:36:58.1082809+03:00

102 % No issues found

Ln: 15, Ch: 23 SPC CRLF UTF-8

Query executed successfully. localhost (17.0 RTM) LAPTOP-H8LGQORynourm NikeSales 00:00:00 Row: 0, Col: 0 0 rows

FACT SALES Dimension Table Creation

This screenshot shows the SQL Server Enterprise Manager interface with a new query window open. The query is for creating a table named 'FactSales' with the following structure:

```
CREATE TABLE dbo.FactSales
(
    Sales_Key INT IDENTITY(1,1) PRIMARY KEY,
    Date_Key INT NOT NULL,
    Customer_Key INT NOT NULL,
    Product_Key INT NOT NULL,
    Region_Key INT NOT NULL,
    Quantity INT,
    Unit_Price_USD DECIMAL(18,2),
    Gross_Sales_USD DECIMAL(18,2),
    Net_Sales_USD DECIMAL(18,2),
    Tax_USD DECIMAL(18,2),
    Shipping_USD DECIMAL(18,2),
    Order_Total_USD DECIMAL(18,2),
    CONSTRAINT PK_FactSales_Date
        FOREIGN KEY (Date_Key)
        REFERENCES dbo.DateSales(Date_Key),
    CONSTRAINT FK_FactSales_Customer
        FOREIGN KEY (Customer_Key)
        REFERENCES dbo.CustomerSales(Customer_Key),
    CONSTRAINT FK_FactSales_Product
        FOREIGN KEY (Product_Key)
        REFERENCES dbo.ProductSales(Product_Key),
    CONSTRAINT FK_FactSales_Region
        FOREIGN KEY (Region_Key)
        REFERENCES dbo.RegionSales(Region_Key)
)
```

The status bar at the bottom indicates the query was executed successfully.

This screenshot shows the SQL Server Enterprise Manager interface with a new query window open. The query is for creating a table named 'FactSales' with the following structure:

```
CREATE TABLE dbo.FactSales
(
    Sales_Key INT IDENTITY(1,1) PRIMARY KEY,
    Date_Key INT NOT NULL,
    Customer_Key INT NOT NULL,
    Product_Key INT NOT NULL,
    Region_Key INT NOT NULL,
    Quantity INT,
    Unit_Price_USD DECIMAL(18,2),
    Gross_Sales_USD DECIMAL(18,2),
    Net_Sales_USD DECIMAL(18,2),
    Tax_USD DECIMAL(18,2),
    Shipping_USD DECIMAL(18,2),
    Order_Total_USD DECIMAL(18,2),
    CONSTRAINT PK_FactSales_Date
        FOREIGN KEY (Date_Key)
        REFERENCES dbo.DateSales(Date_Key),
    CONSTRAINT FK_FactSales_Customer
        FOREIGN KEY (Customer_Key)
        REFERENCES dbo.CustomerSales(Customer_Key),
    CONSTRAINT FK_FactSales_Product
        FOREIGN KEY (Product_Key)
        REFERENCES dbo.ProductSales(Product_Key),
    CONSTRAINT FK_FactSales_Region
        FOREIGN KEY (Region_Key)
        REFERENCES dbo.RegionSales(Region_Key)
)
```

The status bar at the bottom indicates the query was executed successfully.

This screenshot shows the SQL Server Enterprise Manager interface with a query window open. The query is for selecting the top 10 rows from the 'FactSales' table:

```
SELECT TOP 10 *
FROM dbo.FactSales;
```

The results are displayed in a table with the following columns: Sales_Key, Date_Key, Customer_Key, Product_Key, Region_Key, Quantity, Unit_Price_USD, Discount_Rate, Gross_Sales_USD, Net_Sales_USD, Tax_USD, Shipping_USD, and Order_Total_USD. The status bar at the bottom indicates the query was executed successfully.

Sales_Key	Date_Key	Customer_Key	Product_Key	Region_Key	Quantity	Unit_Price_USD	Discount_Rate	Gross_Sales_USD	Net_Sales_USD	Tax_USD	Shipping_USD	Order_Total_USD
1	208	1050	51	1	1	95.00	0.1600	95.00	88.64	3.88	0.00	84.32
2	356	147	14	2	1	185.00	0.0600	185.00	155.10	12.20	0.00	167.30
3	43	277	30	4	2	199.00	0.1500	398.00	338.30	14.47	0.00	352.77
4	213	1599	18	1	1	34.00	0.0000	34.00	30.84	3.18	0.00	34.12
5	15	1099	5	3	1	180.00	0.0000	180.00	160.00	8.00	0.00	168.00
6	229	1395	49	2	2	29.00	0.0000	58.00	58.00	5.05	0.00	63.05
7	277	1719	27	2	2	178.00	0.0000	356.00	256.00	39.31	0.00	395.31
8	351	1569	51	5	1	96.00	0.2200	96.00	74.68	6.29	0.00	81.17
9	43	404	7	5	1	43.00	0.0000	43.00	43.00	2.99	0.00	45.99
10	321	1046	36	2	2	58.00	0.0600	112.00	105.28	6.99	0.00	112.98

Customer Segment Revenue & Order Analysis

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the Object Explorer with the 'NikeSales' database selected. The central pane shows a SQL query being executed. The bottom pane shows the results of the query, which is a table with four columns: Customer_Segment, Total_Revenue, and Total_Orders. The results are sorted by Total_Revenue in descending order.

SQL Query:

```
1 SELECT
2   C.Customer_Segment,
3   SUM(T.Net_Sales_USD) AS Total_Revenue,
4   COUNT(T.Customer_ID) AS Total_Orders
5 FROM dbo.Transactions AS T
6 INNER JOIN dbo.DimCustomer AS C
7   ON T.Customer_ID = C.Customer_ID
8 GROUP BY
9   C.Customer_Segment
10 ORDER BY
11   Total_Revenue DESC;
```

Query Results:

Customer_Segment	Total_Revenue	Total_Orders
Student	4038094.70	30048
Member	4000119.00	35692
Business	3759602.06	33058
Consumer	3679518.06	32870

Query executed successfully. localhost (17.0 RTM) LAPTOP-I6LGGQRynourm NikeSales 00:00:00 Row: 1, Col: 1 4 rows

Star Schema Data Model / ERD Diagram

